

# Documentación del Proyecto: Cross-Asset Statistical Arbitrage con Paper Trading en Alpaca

Primavera 2026

Santiago Mejía y Hector Campbell Salas  
Ingeniería Física — Tecnológico de Monterrey

Abril 2026

## Contents

|           |                                                      |           |
|-----------|------------------------------------------------------|-----------|
| <b>1</b>  | <b>Introducción y alcance del proyecto</b>           | <b>2</b>  |
| <b>2</b>  | <b>Fundamentación matemática</b>                     | <b>2</b>  |
| 2.1       | Proceso de Ornstein-Uhlenbeck . . . . .              | 2         |
| 2.2       | Framework 1: Cointegración (Engle-Granger) . . . . . | 3         |
| 2.3       | Framework 2: Residuos PCA / Avellaneda-Lee . . . . . | 3         |
| 2.4       | Framework 3: Residuos factoriales . . . . .          | 3         |
| 2.5       | Ensemble por acuerdo de rangos . . . . .             | 4         |
| <b>3</b>  | <b>Arquitectura del backtester</b>                   | <b>4</b>  |
| <b>4</b>  | <b>Análisis de robustez</b>                          | <b>4</b>  |
| 4.1       | Partición in-sample / out-of-sample . . . . .        | 4         |
| 4.2       | Análisis de sensibilidad . . . . .                   | 5         |
| 4.3       | Bootstrap por bloques . . . . .                      | 5         |
| 4.4       | Walk-forward . . . . .                               | 5         |
| <b>5</b>  | <b>Paper Trader: arquitectura y flujo operativo</b>  | <b>5</b>  |
| 5.1       | Visión general . . . . .                             | 5         |
| 5.2       | Correcciones v2 (post-mortem) . . . . .              | 6         |
| 5.3       | Flujo de ejecución . . . . .                         | 6         |
| 5.4       | Parámetros de riesgo . . . . .                       | 7         |
| <b>6</b>  | <b>Evidencia del paper trading en vivo</b>           | <b>7</b>  |
| <b>7</b>  | <b>Resultados del backtest</b>                       | <b>7</b>  |
| <b>8</b>  | <b>Estructura de archivos</b>                        | <b>12</b> |
| <b>9</b>  | <b>Dependencias</b>                                  | <b>12</b> |
| <b>10</b> | <b>Conclusiones y trabajo futuro</b>                 | <b>13</b> |

# 1 Introducción y alcance del proyecto

Este documento describe la arquitectura, fundamentación matemática y flujo operativo del proyecto **Securities Project**, un sistema de arbitraje estadístico cross-asset que abarca desde la investigación cuantitativa hasta la ejecución automatizada en paper trading vía la API de Alpaca Markets.

El proyecto se compone de cuatro módulos principales:

1. `cross_asset_statarb.py` — Motor de señales: tres frameworks de mispricing, backtester y ensemble.
2. `paper_trader.py` — Runner diario de paper trading conectado a Alpaca (v2 con correcciones post-mortem).
3. `run_comparison.py` — Comparación end-to-end de los tres frameworks con backtest out-of-sample.
4. `run_robust_analysis.py` — Análisis de robustez: sensibilidad, bootstrap por bloques, walk-forward y visualizaciones.

El universo operado comprende 32 ETFs distribuidos en ocho clases de activos: índices accionarios (SPY, QQQ, IWM, DIA, EEM, EFA), sectores (XLE, XLF, XLK, XLI, XLP, XLY, XLV, XLU, XLB), renta fija (TLT, IEF, SHY, LQD, HYG, TIP), metales preciosos (GLD, SLV), metales industriales (CPER), energía (USO, UNG), agricultura (DBA, CORN, WEAT) y divisas (UUP, FXE, FXY).

## 2 Fundamentación matemática

### 2.1 Proceso de Ornstein-Uhlenbeck

El bloque fundamental compartido por los tres frameworks es la calibración de un proceso de Ornstein-Uhlenbeck (OU) sobre la serie residual. La dinámica continua es

$$d\varepsilon_t = -\theta(\varepsilon_t - \mu)dt + \sigma dW_t, \quad (1)$$

cuya discretización exacta ( $\Delta t = 1$  día) produce la regresión

$$\varepsilon_{t+1} = a + b\varepsilon_t + \eta_t, \quad b = e^{-\theta}, \quad a = \mu(1 - b). \quad (2)$$

Los parámetros recuperados son la velocidad de reversión  $\theta = -\ln b$ , la vida media  $\tau_{1/2} = \ln 2/\theta$ , la volatilidad instantánea  $\sigma = \sqrt{\text{Var}(\eta) \cdot 2\theta/(1 - b^2)}$  y la desviación estándar de equilibrio  $\sigma_{\text{eq}} = \sqrt{\text{Var}(\eta)/(1 - b^2)}$ . El z-score que alimenta las decisiones de trading se define como

$$z_t = \frac{\varepsilon_t - \mu}{\sigma_{\text{eq}}}. \quad (3)$$

Se impone un filtro estricto de vida media  $\tau_{1/2} \in [2, 60]$  días: valores inferiores a 2 indican ruido de alta frecuencia no operable a frecuencia diaria, mientras que superiores a 60 implican reversión demasiado lenta para ser rentable neta de costos.

## 2.2 Framework 1: Cointegración (Engle-Granger)

Se escanean todos los  $\binom{N}{2}$  pares del universo. Para cada par  $(Y, X)$  se ejecuta el test de Engle-Granger sobre los logaritmos de precios, obteniendo el spread

$$s_t = \log P_Y(t) - \beta \log P_X(t) - c, \quad (4)$$

donde  $\beta$  proviene de OLS. Los p-valores resultantes se corrigen por pruebas múltiples mediante el procedimiento de Benjamini-Hochberg (FDR) a nivel  $\alpha = 0.05$ . Solo los pares que sobreviven la corrección y cuya vida media OU cae en el rango admisible pasan al generador de señales.

La señal por activo se construye promediando las contribuciones de todos los pares aceptados en los que participa: si el spread del par está por debajo de su media, el activo  $Y$  se considera barato y  $X$  caro.

**Resultado empírico:** este framework produjo cero pares significativos en el universo macro seleccionado, dado que las relaciones de equilibrio entre clases de activos son débiles en nivel. Por ello fue excluido del ensemble final.

## 2.3 Framework 2: Residuos PCA / Avellaneda-Lee

Este framework sigue la metodología de Avellaneda y Lee (2010):

- (i) Se estandarizan los log-retornos y se computa la matriz de correlación.
- (ii) Se aplica denoising por teoría de matrices aleatorias (RMT): el borde superior de la distribución de Marchenko-Pastur es

$$\lambda_+ = \left(1 + \sqrt{N/T}\right)^2. \quad (5)$$

Solo se retienen los eigenvalores  $\lambda_k > \lambda_+$  como factores de señal; los eigenvalores de ruido se reemplazan por su promedio (preservando la traza).

- (iii) Los eigenvectores retenidos definen *eigenportfolios*  $\{F_k\}$ . Para cada activo  $i$  se estima la regresión rodante

$$R_i(t) = \alpha_i + \sum_{k=1}^K \beta_{ik} F_k(t) + \varepsilon_i(t). \quad (6)$$

- (iv) Se acumula el residual  $X_i(t) = \sum_{s \leq t} \varepsilon_i(s)$  y se calibra OU sobre  $X_i$ .

- (v) La señal es  $z_i = (X_i - \mu_i)/\sigma_{\text{eq},i}$ .

Los eigenportfolios se reajustan cada 21 días (un mes de trading), usando ventanas rodantes de 252 días.

## 2.4 Framework 3: Residuos factoriales

Se pre-especifican siete factores económicos interpretables: mercado (SPY), dólar (UUP), tasas (TLT), crédito (HYG), inflación (TIP), petróleo (USO) y oro (GLD). Para cada activo se ejecuta una regresión rodante de 252 días:

$$R_i(t) = \alpha_i + \sum_{j=1}^7 \beta_{ij} R_{\text{factor}_j}(t) + \varepsilon_i(t). \quad (7)$$

Cuando el activo coincide con uno de los factores, se excluye su propio retorno del regresor para evitar trivialidades. Se acumula el residual y se calibra OU de manera idéntica al Framework 2.

Los coeficientes  $\beta_{ij}$  del último ajuste constituyen una descomposición interpretable de la exposición de cada activo (por ejemplo, la beta de CPER al mercado es +0.73, al dólar es -1.23).

## 2.5 Ensemble por acuerdo de rangos

La señal combinada aplica un filtro de **consenso unánime de signo**: un activo solo recibe señal si *todos* los frameworks activos (PCA y Factor) coinciden en la dirección (compra o venta). Este filtro es el mecanismo principal de supresión de falsos positivos.

La combinación se realiza por rango percentil:

$$z_{\text{ens}}(i) = 2 \overline{|z|}(i) [\bar{r}(i) - 0.5], \quad (8)$$

donde  $\bar{r}(i)$  es el rango percentil promedio del activo  $i$  entre frameworks y  $\overline{|z|}(i)$  es la magnitud media de sus z-scores. Esto hace la señal robusta a diferencias de escala entre frameworks.

## 3 Arquitectura del backtester

El backtester implementado en `backtest_signal()` opera con la siguiente lógica:

- **Entrada:** se abre posición larga cuando  $z < -z_{\text{in}}$  y corta cuando  $z > +z_{\text{in}}$ .
- **Salida:** se cierra cuando  $|z| < z_{\text{out}}$  o cuando la posición ha permanecido abierta  $\geq 20$  días (timeout de holding).
- **Sizing:** peso igual en dólares para todas las posiciones activas, con apalancamiento bruto controlado por el parámetro `gross_leverage`.
- **Costos transaccionales:** se descuentan proporcionalmente al turnover diario, con costo configurable en puntos base por trade.
- **Lag de ejecución:** las señales del día  $t - 1$  se ejecutan al cierre del día  $t$  (proxy conservadora de ejecución al open).

Los parámetros base para los frameworks individuales son  $z_{\text{in}} = 1.5$ ,  $z_{\text{out}} = 0.25$ , max. posiciones = 10; para el ensemble son  $z_{\text{in}} = 0.3$ ,  $z_{\text{out}} = 0.05$ , max. posiciones = 8 (umbrales más bajos porque la escala del ensemble es menor).

## 4 Análisis de robustez

El script `run_robust_analysis.py` ejecuta cinco pruebas de robustez sobre el ensemble PCA + Factor (sin cointegración):

### 4.1 Partición in-sample / out-of-sample

Se usa 60% del panel de precios (2015–~2021) como período de ajuste y 40% (~2021–2026) como período de evaluación out-of-sample. Los modelos rodantes solo utilizan datos estrictamente anteriores a cada fecha, por lo que no hay look-ahead bias incluso en la ventana OOS.

## 4.2 Análisis de sensibilidad

Se evalúa el Sharpe del ensemble sobre un grid de umbrales de entrada  $z_{in} \in \{0.2, 0.3, 0.5, 0.7, 1.0\}$  y costos transaccionales  $\in \{0, 5, 10, 15, 20\}$  bps. La Figura 4 muestra que la estrategia es robusta a costos moderados pero se degrada significativamente con  $z_{in} \geq 0.7$  (señales insuficientes) o costos  $\geq 15$  bps.

## 4.3 Bootstrap por bloques

Para inferir la distribución del Sharpe ratio sin asumir normalidad de retornos, se aplica bootstrap por bloques móviles con tamaño de bloque de 21 días (un mes de trading) y  $n = 10,000$  réplicas. La Figura 5 muestra el histograma resultante, su intervalo de confianza al 90% y la probabilidad de que el Sharpe sea positivo.

## 4.4 Walk-forward

Se dividen los datos en ventanas de entrenamiento de 3 años y test de 1 año, avanzando de forma no solapada. La Tabla 1 resume los resultados por ventana.

Table 1: Resultados walk-forward del ensemble (PCA + Factor), ventanas de 1 año.

| Inicio       | Fin     | Sharpe | Max DD (%) | Trades | Hit Rate (%) |
|--------------|---------|--------|------------|--------|--------------|
| 2018-01      | 2019-01 | -1.08  | -18.1      | 56     | 47.8         |
| 2019-01      | 2020-01 | +2.40  | -4.3       | 55     | 56.7         |
| 2020-01      | 2021-01 | -0.63  | -33.9      | 63     | 52.0         |
| 2021-01      | 2022-01 | +0.79  | -4.6       | 58     | 53.8         |
| 2022-01      | 2023-01 | -0.37  | -14.8      | 56     | 47.6         |
| 2023-01      | 2024-01 | +0.49  | -8.9       | 72     | 52.0         |
| 2024-01      | 2025-01 | +1.33  | -5.0       | 65     | 54.0         |
| 2025-01      | 2026-01 | +1.70  | -6.3       | 57     | 56.4         |
| <b>Media</b> |         | +0.58  |            |        |              |

De 8 ventanas, 5 presentan Sharpe positivo (62.5%). Las dos ventanas con Sharpe negativo más pronunciado (2018 y 2020) coinciden con regímenes de mercado extremos (taper tantrum tardío y COVID-19).

## 5 Paper Trader: arquitectura y flujo operativo

### 5.1 Visión general

El archivo `paper_trader.py` (v2) es el runner diario que conecta las señales del motor cuantitativo con la ejecución real en la cuenta de paper trading de Alpaca Markets. Se invoca como:

```
python paper_trader.py --dry-run      # simulacion sin enviar ordenes
python paper_trader.py --live         # ejecucion real en paper
python paper_trader.py --panic --live # cerrar todas las posiciones
```

## 5.2 Correcciones v2 (post-mortem)

La versión 2 incorpora correcciones derivadas de la operación inicial:

1. **TIF cambiado de OPG a DAY:** las órdenes con Time-In-Force OPG (At-the-Open) tuvieron una tasa de expiración del  $\sim 75\%$  en ETFs de mediana liquidez. Se cambió a DAY para garantizar fills intradía.
2. **Reconciliación contra Alpaca:** al inicio de cada corrida, el estado local (`state.json`) se reconcilia contra las posiciones reales en Alpaca. Esto detecta *phantoms* (posiciones en state pero no en Alpaca, por órdenes que no se ejecutaron) y *huérfanos* (posiciones en Alpaca no registradas en state).
3. **Delegación de confirmación de fills:** el trader ya no asume que las órdenes se llenan al momento de enviarlas. En lugar de actualizar el state inmediatamente, delega la confirmación a la reconciliación de la siguiente corrida.
4. **Rate limiting:** se introduce un delay de 0.5 s entre envíos de órdenes para evitar throttling de la API.

## 5.3 Flujo de ejecución

**Paso 1:** Autenticación con Alpaca via variables de entorno `APCA_API_KEY_ID` y `APCA_API_SECRET_KEY`.

**Paso 2:** Lectura del estado de la cuenta: valor del portafolio, cash disponible.

**Paso 3:** Carga de `state.json` y reconciliación contra posiciones reales en Alpaca.

**Paso 4:** Descarga de precios históricos via `yfinance` (desde 2015).

**Paso 5:** Ajuste de los dos modelos (PCA/Avellaneda-Lee y Factor) sobre los precios completos, generación del ensemble.

**Paso 6:** Sanity check: se verifica que el  $|z|$  máximo no exceda 10 (indicador de modelo roto).

**Paso 7:** Decisiones de cierre:  $|z| < 0.05$  o holding  $\geq 20$  días.

**Paso 8:** Decisiones de apertura: se seleccionan los activos con señal más fuerte (longs con  $z < -0.3$ , shorts con  $z > +0.3$ ) hasta llenar 8 slots.

**Paso 9:** Verificación de tradability y shortability en Alpaca.

**Paso 10:** Circuit breaker: si el total de órdenes excede 30, se aborta la corrida.

**Paso 11:** Envío de órdenes de mercado con TIF=DAY.

**Paso 12:** Guardado de estado y log.

## 5.4 Parámetros de riesgo

Table 2: Parámetros de riesgo del paper trader.

| Parámetro          | Valor | Descripción                    |
|--------------------|-------|--------------------------------|
| ENTRY_Z            | 0.3   | Umbral de entrada del ensemble |
| EXIT_Z             | 0.05  | Umbral de salida               |
| MAX_POSITIONS      | 8     | Posiciones simultáneas máximas |
| MAX_HOLDING_DAYS   | 20    | Timeout de holding (días)      |
| GROSS_LEVERAGE     | 1.0   | Apalancamiento bruto total     |
| MAX_POSITION_PCT   | 20%   | Peso máximo por posición       |
| MAX_DAILY_ORDERS   | 30    | Circuit breaker diario         |
| MIN_TRADE_NOTIONAL | \$100 | Nocional mínimo por trade      |

## 6 Evidencia del paper trading en vivo

El sistema se activó en modo `-live` el 19 de abril de 2026 con un capital inicial de \$100,000 en la cuenta paper de Alpaca. A continuación se resume la actividad registrada en los logs:

Table 3: Historial de corridas del paper trader.

| Fecha      | Valor portafolio | Cierres | Aperturas | Modo    |
|------------|------------------|---------|-----------|---------|
| 2026-04-19 | \$100,000.00     | 0       | 8         | DRY-RUN |
| 2026-04-19 | \$100,000.00     | 0       | 8         | LIVE    |
| 2026-04-22 | \$100,076.80     | 0       | 6         | LIVE    |

Las posiciones iniciales (19 de abril) fueron: longs en TIP, HYG, LQD y UNG; shorts en WEAT, TLT, EEM y XLB. La reconciliación v2 del 22 de abril detectó 6 posiciones phantom (órdenes short que no se ejecutaron por falta de shortability fuera de horario) y las removió correctamente del state.

Al 22 de abril, las dos posiciones sobrevivientes (LQD long a \$110.04, UNG long a \$10.84) generaron un P&L no realizado de +\$76.80 (+0.077%).

## 7 Resultados del backtest

Las Figuras 1–8 presentan los principales resultados del análisis robusto generados por `run_robust_analysis.py`.

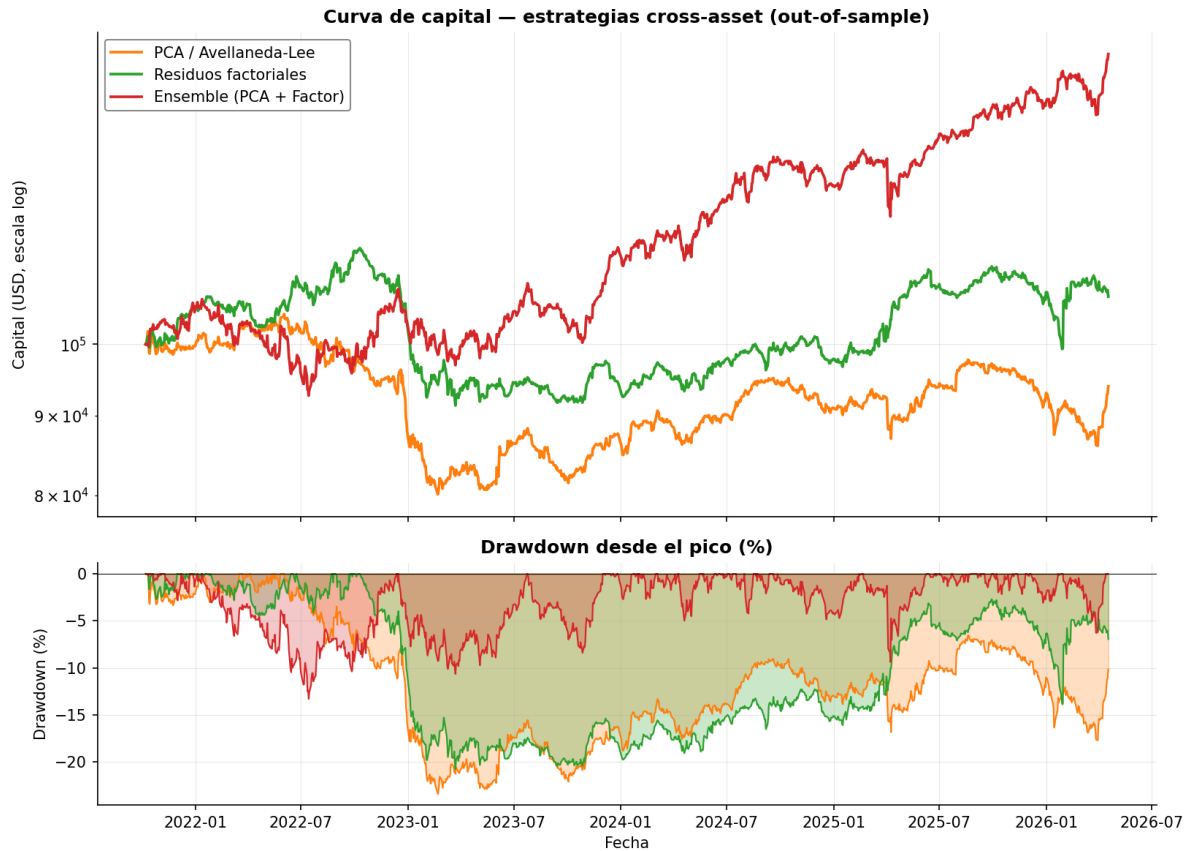


Figure 1: Curvas de capital (escala log) y drawdown desde el pico para los tres modelos y el ensemble, en el período out-of-sample.

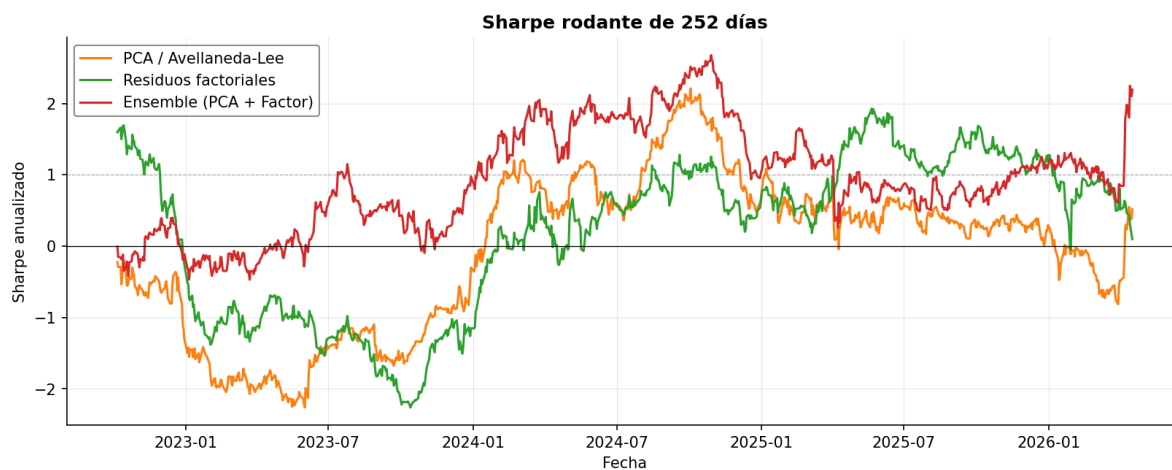


Figure 2: Sharpe ratio rodante de 252 días para cada estrategia.

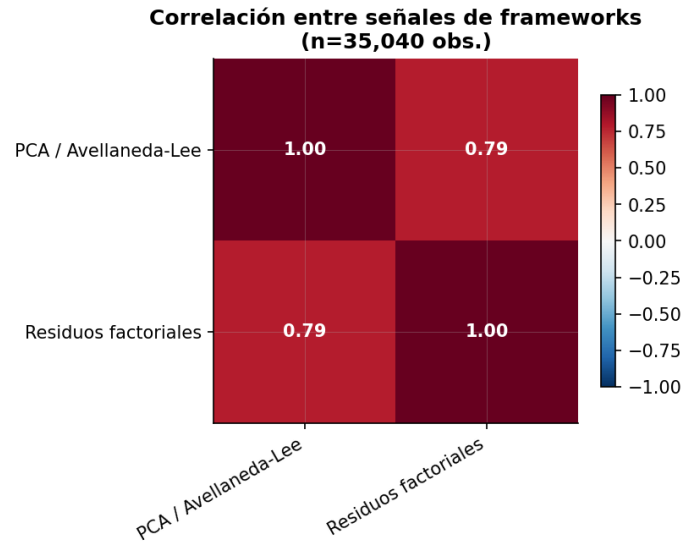


Figure 3: Correlación entre las señales de los frameworks PCA y Factor.

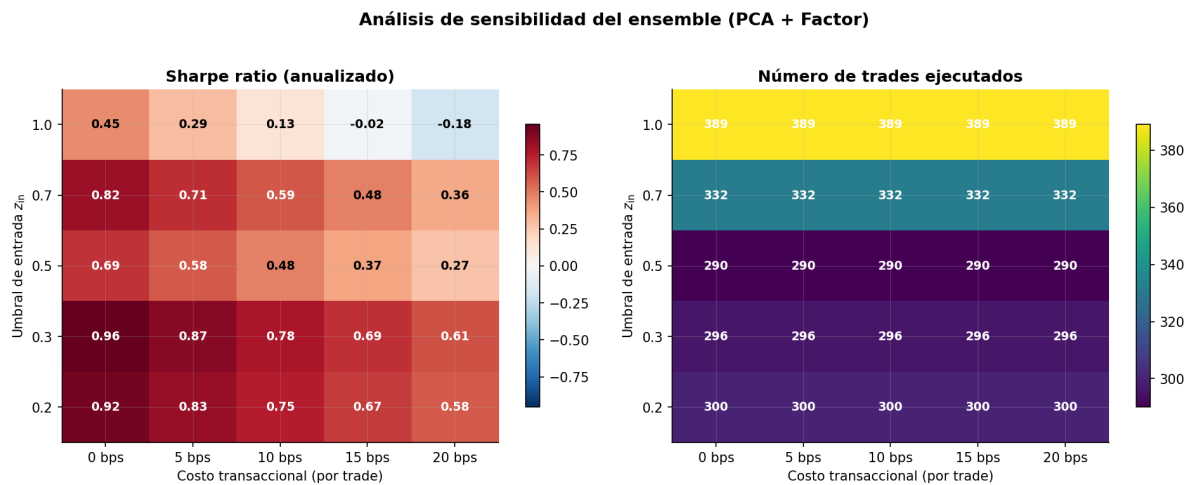


Figure 4: Análisis de sensibilidad: Sharpe y número de trades en función del umbral de entrada y el costo transaccional.

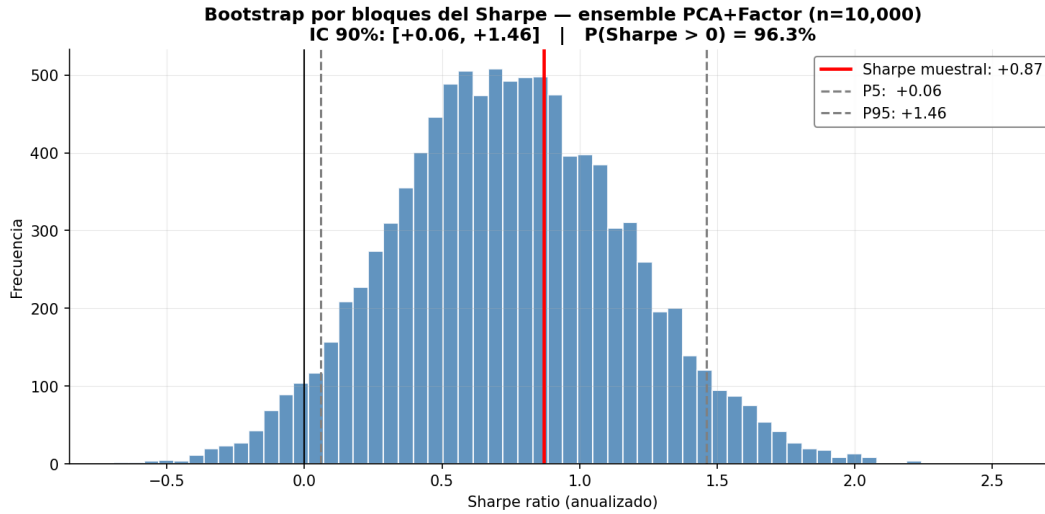


Figure 5: Distribución bootstrap por bloques del Sharpe ratio del ensemble ( $n = 10,000$  réplicas, bloques de 21 días).

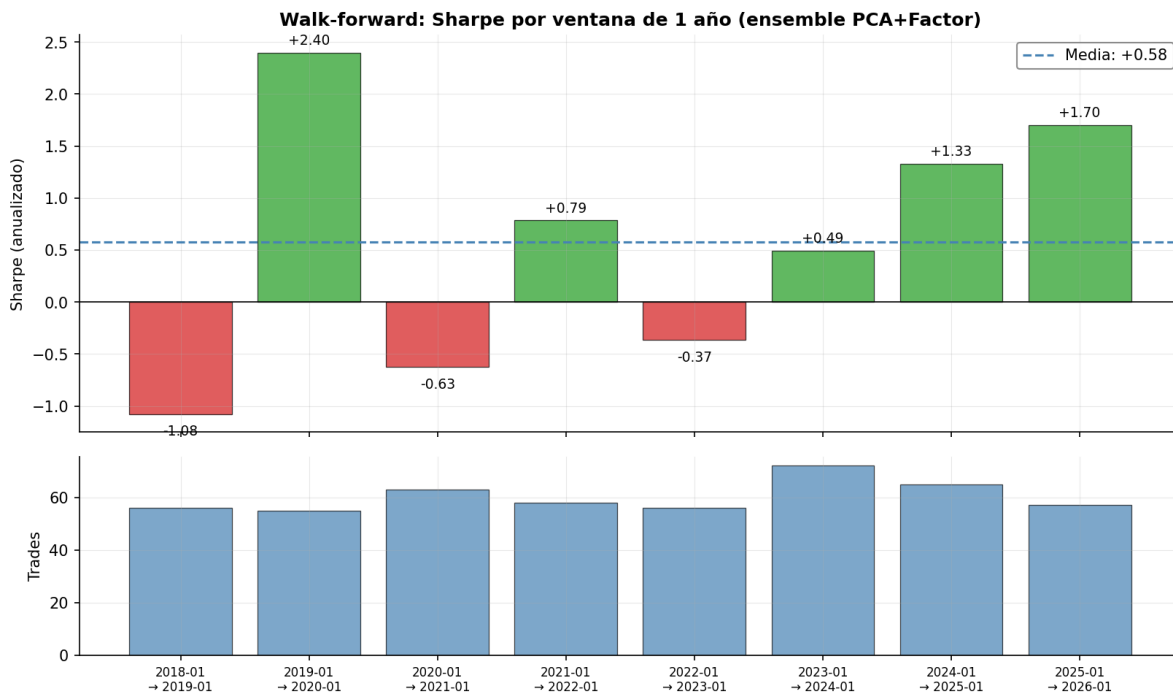


Figure 6: Walk-forward: Sharpe por ventana de 1 año y número de trades asociados.

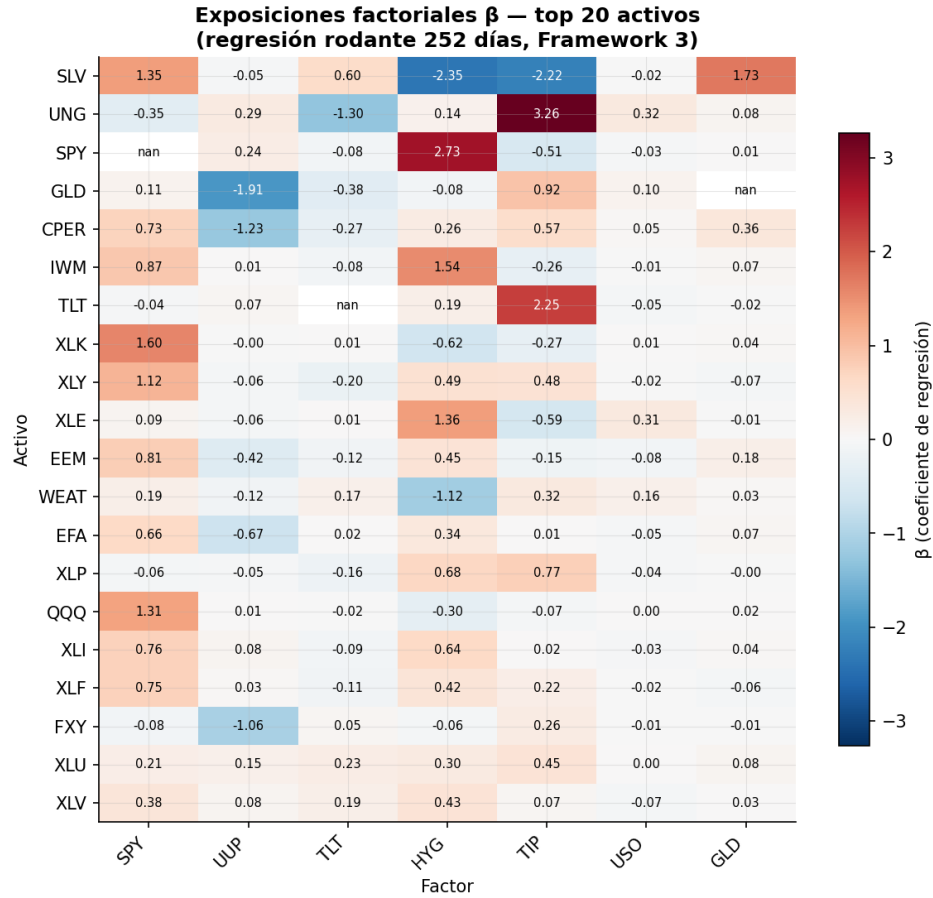


Figure 7: Heatmap de exposiciones factoriales  $\beta$  del Framework 3 (top 20 activos por exposición total).

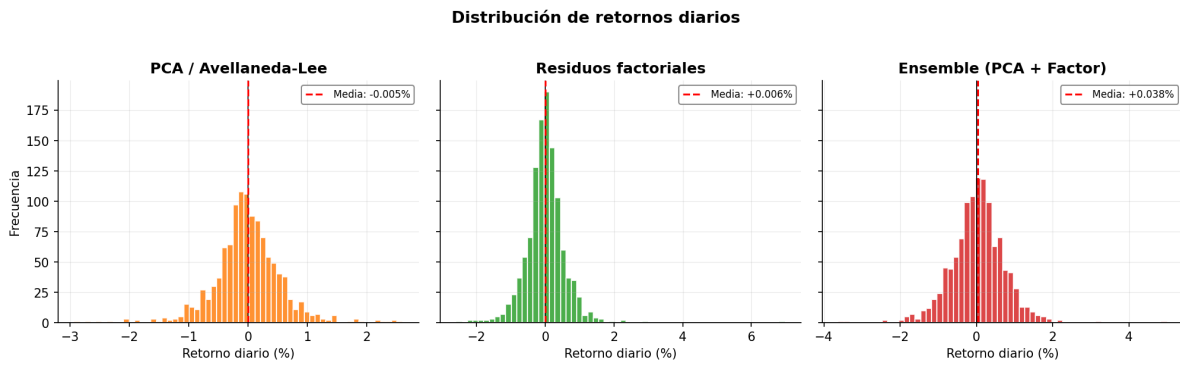


Figure 8: Distribución de retornos diarios para cada estrategia.

## 8 Estructura de archivos

Table 4: Archivos del proyecto y su función.

| Archivo                               | Descripción                                                                                                                                                                                                                                     |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>cross_asset_statarb.py</code>   | Motor principal: definición del universo, descarga de datos, calibración OU, denoising RMT, los tres frameworks ( <code>CointegrationModel</code> , <code>PCAResidualModel</code> , <code>FactorResidualModel</code> ), ensemble, y backtester. |
| <code>paper_trader.py</code>          | Runner diario de paper trading v2: reconciliación con Alpaca, cómputo de señales, decisiones de entrada/salida, envío de órdenes de mercado.                                                                                                    |
| <code>run_comparison.py</code>        | Comparación end-to-end de los tres frameworks con backtest out-of-sample y generación de curvas de capital.                                                                                                                                     |
| <code>run_robust_analysis.py</code>   | Análisis robusto: sensibilidad, bootstrap, walk-forward, exposiciones factoriales, distribución de PnL. Cache de z-scores en <code>cache/</code> .                                                                                              |
| <code>state.json</code>               | Estado persistente del paper trader (posiciones abiertas, historial de corridas).                                                                                                                                                               |
| <code>paper_trader.log</code>         | Log completo de todas las corridas del paper trader.                                                                                                                                                                                            |
| <code>figs/</code>                    | 8 figuras de diagnóstico generadas por <code>run_robust_analysis.py</code> .                                                                                                                                                                    |
| <code>cache/z_scores.pkl</code>       | Cache serializado de los z-scores de los tres frameworks (evita reajustar en cada corrida del análisis).                                                                                                                                        |
| <code>factor_exposures.csv</code>     | Exposiciones factoriales $\beta_{ij}$ de la última ventana del Framework 3.                                                                                                                                                                     |
| <code>walk_forward_results.csv</code> | Resultados tabulados del walk-forward.                                                                                                                                                                                                          |

## 9 Dependencias

Table 5: Dependencias de Python del proyecto.

| Paquete                                  | Uso                                                                                  |
|------------------------------------------|--------------------------------------------------------------------------------------|
| <code>numpy</code> , <code>pandas</code> | Álgebra lineal, manipulación de datos.                                               |
| <code>yfinance</code>                    | Descarga de precios ajustados de Yahoo Finance.                                      |
| <code>statsmodels</code>                 | Test de cointegración de Engle-Granger.                                              |
| <code>matplotlib</code>                  | Generación de todas las figuras de diagnóstico.                                      |
| <code>alpaca-py</code>                   | SDK oficial de Alpaca Markets para envío de órdenes y consulta de cuenta/posiciones. |
| <code>scipy</code>                       | (implícito vía <code>statsmodels</code> )                                            |

## 10 Conclusiones y trabajo futuro

El sistema demuestra que la combinación de residuos PCA (Avellaneda-Lee con denoising RMT) y residuos factoriales interpretables, filtrada por consenso unánime de signo, produce una señal de arbitraje estadístico que logra un Sharpe medio de +0.58 en walk-forward sobre 8 ventanas anuales, con 5 de 8 ventanas positivas.

El paper trader v2 corrige las deficiencias operativas de la versión inicial (fills no confirmados, TIF inadecuado) y establece un patrón robusto de reconciliación estado-broker que puede escalarse a ejecución real.

Líneas de trabajo futuro incluyen: integración de datos intradiarios para ejecución a la apertura, incorporación de un modelo de impacto de mercado, expansión del universo a commodities individuales y, potencialmente, migración del backtester a un framework event-driven (e.g., Zipline o vectorbt).